

YZ 103 · INTRODUCTION TO PROGRAMMING

Introduction to Computers and Java

Week 1 — How computers run the programs we write, and why Java was built the way it was.

What You'll Be Able to Do

- 1 Explain how computer hardware and software work together
- 2 Describe how a high-level program becomes something a machine can run
- 3 Explain why Java compiles to byte-code and what the JVM does
- 4 Recognize the core ideas of object-oriented programming
- 5 Name the three kinds of programming errors, and where each is caught

Hardware and Software

Hardware

The tangible, physical parts of a computer system.

- Input devices — keyboard, mouse
- Output devices — screen, printer
- A processor (CPU)
- Main memory and auxiliary memory

Software

Programs — sets of instructions for the computer to follow.

- Everything from an OS to a game
- Written by a programmer
- Runs on top of the hardware
- Understanding hardware helps you understand software

The Core Hardware Components



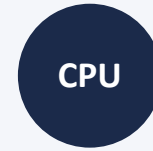
Input Devices

Keyboard, mouse, microphone —
how data enters the system



Output Devices

Screen, printer, speakers — how
results reach the user



The Processor

Executes simple instructions,
extremely fast — the "chip"



Memory

Two kinds: main memory
(working) and auxiliary memory
(storage)

Two Kinds of Memory

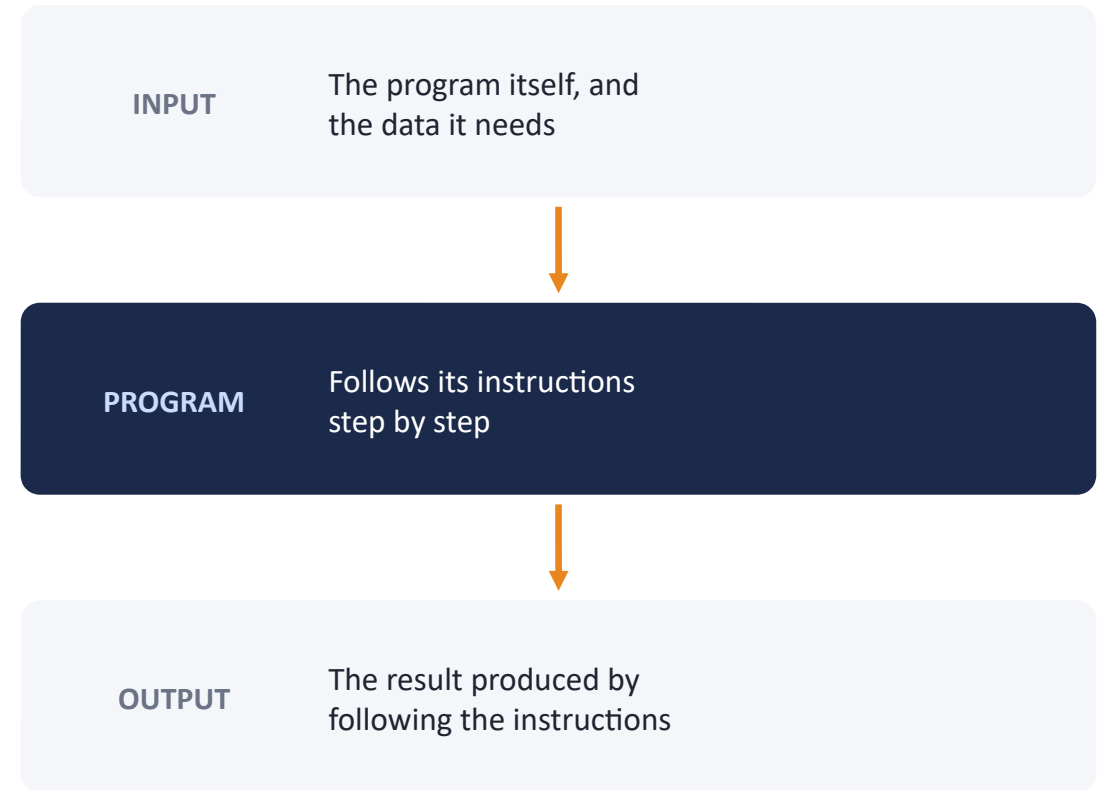
	Main Memory (RAM)	Auxiliary Memory
Holds	Running program, its data, intermediate results	Files, programs — long-term storage
Speed	Fast — working memory	Slower, but persistent
Lifetime	Volatile (cleared when off)	Non-volatile (survives power-off)
Examples	8 GB, 16 GB of RAM	Hard disk, SSD, flash drive
Measured in	Megabytes / Gigabytes	Gigabytes / Terabytes

A byte = 8 bits. Each byte in main memory has a numbered address.

What Is a Program?

A **program** is a set of instructions for a computer to follow. Running (or executing) a program means having the computer follow those instructions.

Email, word processors, video games, bank ATMs — you use programs almost every hour of the day.



From Source Code to Machine Code

High-level languages

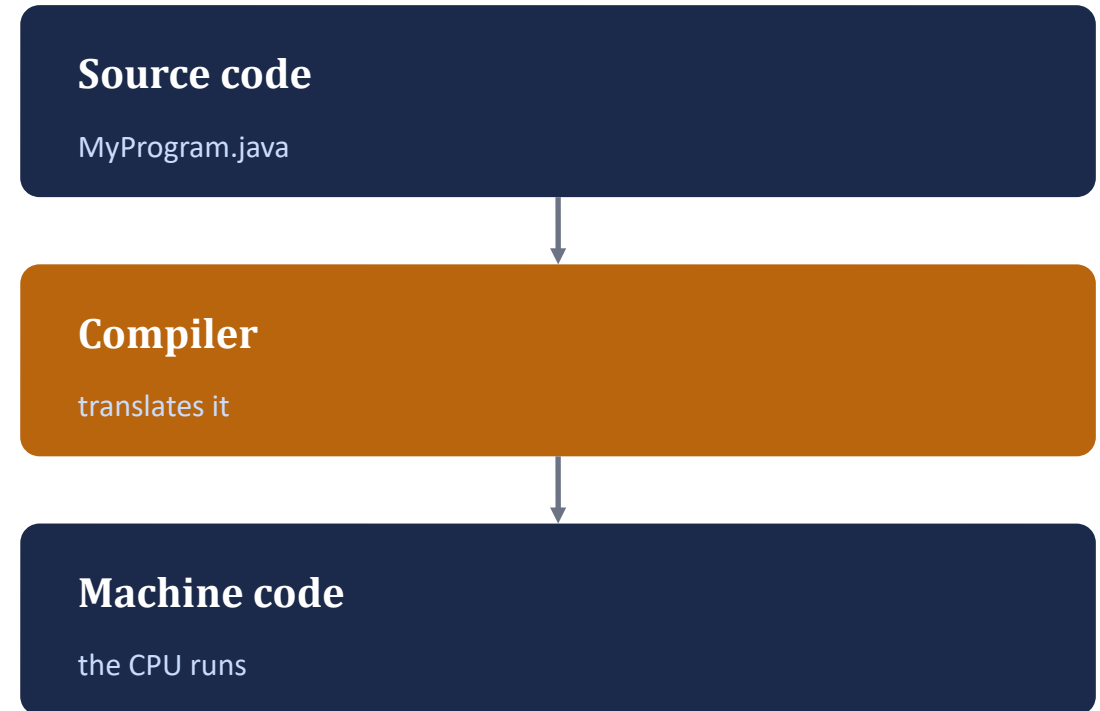
Java, C#, C++, Python, Ruby — relatively easy for humans to read and write.

The problem

Computer hardware only understands low-level (machine) language.

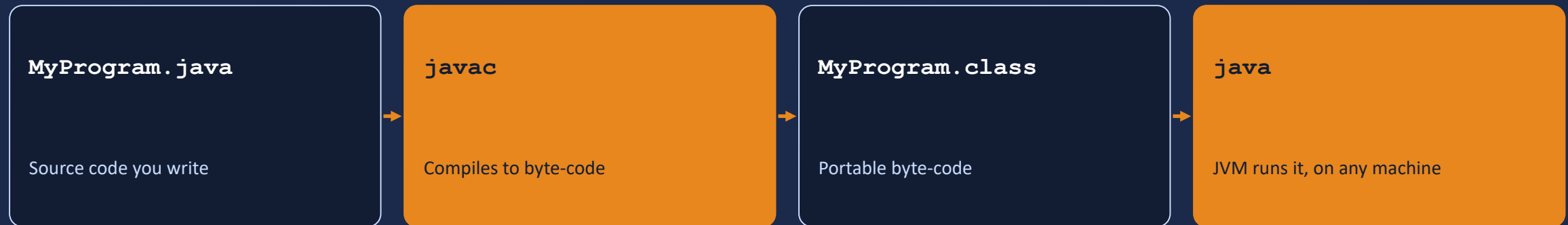
Compiler

Translates a source program (high-level) into an object program (machine/assembly). Usually one compiler per language, per machine, per OS.



Byte-Code and the Java Virtual Machine

Java doesn't compile straight to one machine's language. It compiles to **byte-code** — the machine language of a hypothetical computer called the **Java Virtual Machine (JVM)**.



Result: “Write once, run anywhere.” Byte-code can travel over the Internet and run on any machine with a JVM — no recompiling.

A Brief History of Java

1991

James Gosling and Sun Microsystems begin designing a language for smart home appliances.

1991-93

Different appliances use different chips — the team designs an intermediate language common to all of them.

1994

Gosling realizes the same idea is ideal for a web browser that runs programs over the Internet.

1995

Sun releases HotJava, and the language we know as Java is born.

Object-Oriented Programming

Our world is made of objects — people, cars, cities, bank accounts. Objects perform actions that affect themselves and other objects. OOP treats a program as a collection of interacting objects.



Object

An individual “thing” with its own data — a specific car, a specific student.



Class

The blueprint for a kind of object — all objects of a class share methods and kinds of data.



Method

An action an object can perform.

Three Pillars of OOP

1

Encapsulation

Data and methods are bundled together, but only part is exposed. Also called information hiding.

You use a car's pedals and wheel — not its fuel injectors.

2

Polymorphism

The same instruction takes different forms depending on the object using it.

“Take time to recreate” means something different to different people.

3

Inheritance

Classes are organized in a hierarchy; lower classes inherit traits from classes above them.

A Sedan inherits everything a Vehicle already has.

What Is an Algorithm?

An algorithm is a set of instructions for solving a problem — expressed completely and precisely, usually in English or pseudocode. Once you have one, turning it into Java is usually the easy part.

Example — Total Cost of All Items

Write 0 on the whiteboard.

For each item on the list:

- add its cost to the number on the whiteboard,
- replace the number with the result.

Announce the number on the whiteboard as the answer.

Notice: no Java syntax yet — just a precise, unambiguous procedure.

Three Kinds of Errors

Syntax Errors

Grammatical mistakes in the code.

Caught by the compiler — it prints an error message.

e.g. a period where a comma belongs

Runtime Errors

Detected while the program runs, not at compile time.

The program stops and prints an error message.

e.g. dividing by zero

Logic Errors

Not caught by the compiler or at runtime — the program just gives the wrong answer.

Caught only by careful testing.

e.g. using +23 instead of +32 in a formula

A First Look at Java Syntax

```
public class FirstProgram
{
    public static void main(String[] args)
    {
        System.out.println("Programming is fun!");
    }
}
```

Statement

An instruction; ends with a semicolon.

Object.method(args)

System.out is an object; println is a method you call on it.

Class

Every Java program is built from one or more classes.

main method

public static void main(String[] args) — where a program starts running.

Quick Recap, and What's Coming Up

SO FAR



Hardware runs software; the CPU executes simple instructions extremely fast.



High-level code is translated by a compiler; Java compiles to portable byte-code run by the JVM.



OOP organizes programs as interacting objects: encapsulation, polymorphism, inheritance.



Bugs come in three flavors — syntax, runtime, logic — caught at different stages.

COMING UP IN THIS SESSION

1

Binary numbers

How computers really store and convert numbers.

2

Algorithms & flowcharts

Designing a solution before writing any code.

3

Variables, decisions, loops

The three building blocks, drawn and traced by hand.

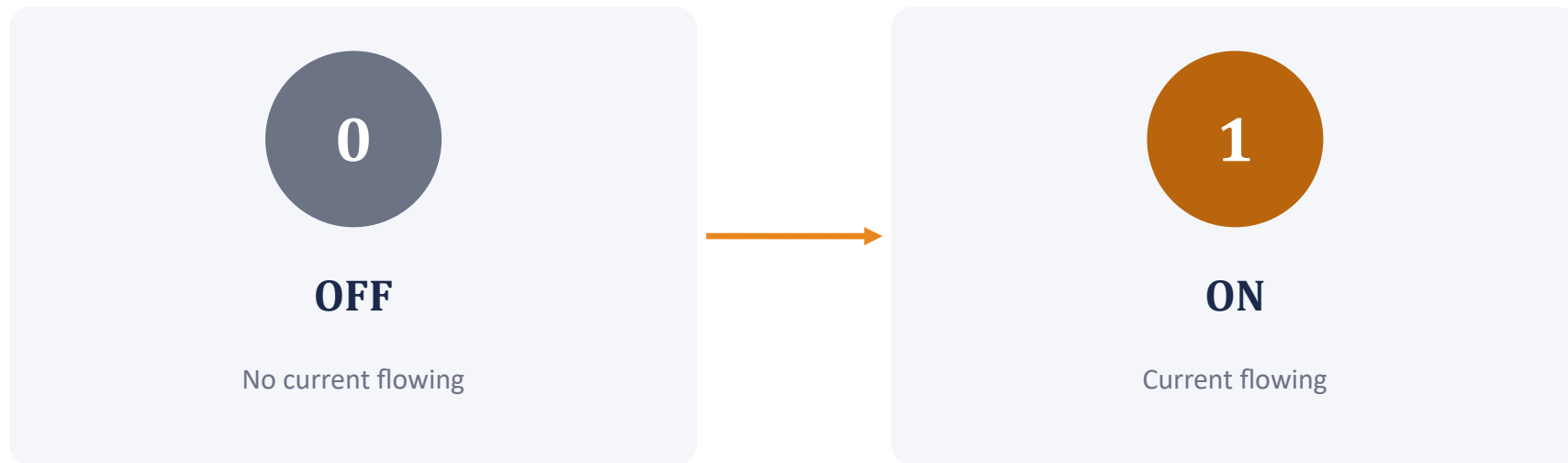
4

Pseudocode → Java

Bridging what we've drawn into real syntax next time.

Why Only 0 and 1?

A transistor — the tiny switch inside a chip — is most reliable in exactly **two states**: current flowing, or not. Building hardware that reliably tells 10 different voltage levels apart is hard; telling **on** from **off** is easy and fast.



Every number, letter, image, and program is ultimately encoded as a sequence of these two digits — bits.

The Same Idea, a Different Base

In decimal, each position is worth **10×** the position to its right. In binary, each position is worth **2×** the position to its right — same idea, smaller multiplier.

Decimal
(base 10)

1000

100

10

1

Binary
(base 2)

8

4

2

1

Decimal place values: $10^3, 10^2, 10^1, 10^0 = 1000, 100, 10, 1$

Binary place values: $2^3, 2^2, 2^1, 2^0 = 8, 4, 2, 1$

Converting Decimal to Binary

Divide by 2 repeatedly. Each remainder is one binary digit — read the remainders bottom to top.

$$13 \div 2 = 6 \quad \text{remainder } 1$$

$$6 \div 2 = 3 \quad \text{remainder } 0$$

$$3 \div 2 = 1 \quad \text{remainder } 1$$

$$1 \div 2 = 0 \quad \text{remainder } 1$$

Read bottom → top

1 1 0 1

13 in decimal is 1101 in binary.

Check: $8+4+0+1 = 13$ ✓

Converting Binary to Decimal

Multiply each digit by its place value, then add everything up.

1	0	1	1
$\times 8$	$\times 4$	$\times 2$	$\times 1$
8	0	2	1
+ + +			
1011 in binary = 8 + 0 + 2 + 1 = 11 in decimal			

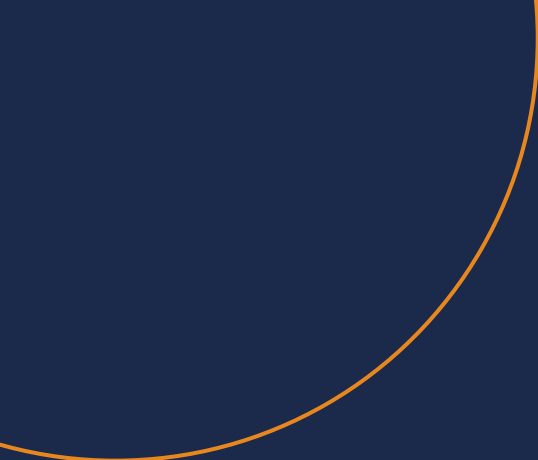
Key takeaway

8 bits = 1 byte

1 byte stores 0 to 255 (256 values)

Same idea scales up to bytes, kilobytes, megabytes...

Try it yourself: what's 1010 in decimal? What's 20 in binary?

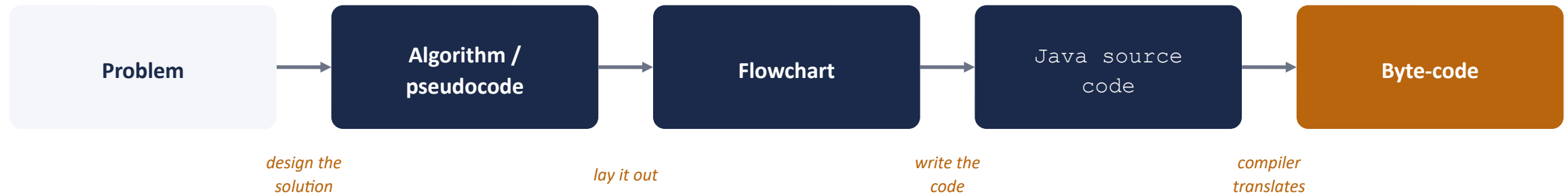


NEW UNIT

From Algorithms to Flowcharts

Before we write a single line of Java, we'll draw the logic itself — sequence, decision, and repetition — as flowcharts. Variables, if, and loops start here, on paper.

How Do We Get a Computer to Do Something?



- A computer does nothing at all unless it is told to.
- Everything it must do has to be spelled out step by step, completely and without ambiguity.

A COMMON MISTAKE

Do not start writing a program at the keyboard. Code for a problem whose algorithm hasn't been worked out usually has to be thrown away and written again.

The Four Symbols You Need



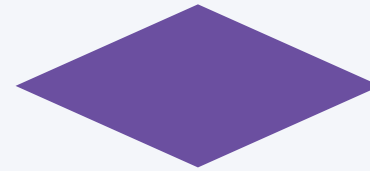
Start / End

Marks where the flowchart begins or ends. Every flowchart has exactly one Start.



Process

A step that does something — a calculation, or storing a value in a variable.



Decision

A yes/no question. Flow splits into two paths depending on the answer.



Input / Output

Getting data from the user, or displaying a result on the screen.

Arrows (flow lines) connect the symbols and show which direction the logic moves.

What Makes a Valid Algorithm?

An algorithm is an ordered, unambiguous chain of steps describing everything that must be done to carry out a task. To count as one, it has to satisfy four properties.

1

Definiteness

Every step admits exactly one reading — no room for guessing what it means.

2

Finiteness

It must end after finitely many steps — a real recipe, not one that runs forever.

3

Output

It must produce at least one result — otherwise nothing was actually solved.

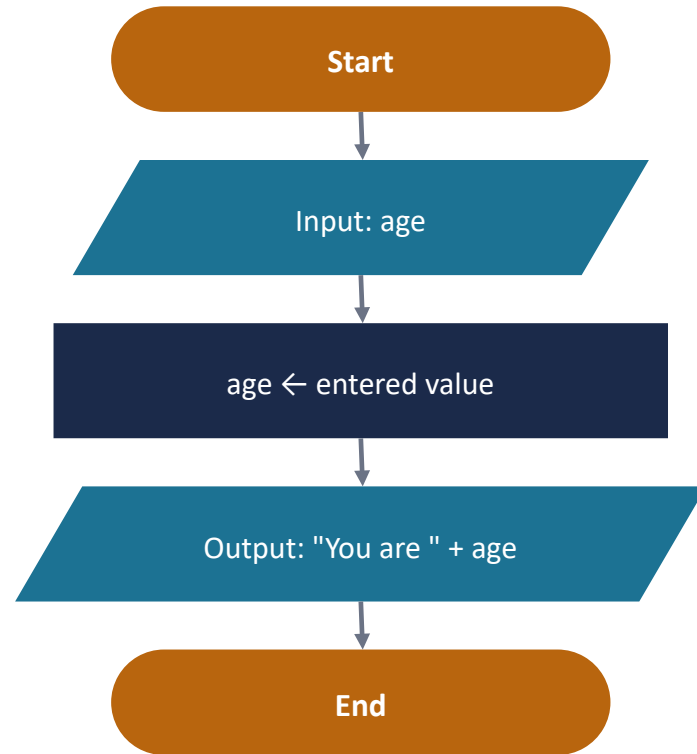
4

Effectiveness

Every step must be doable in finite time with the resources at hand.

One problem admits many algorithms. They can all be correct and still differ enormously in efficiency — more on that shortly.

A Labeled Box for a Value



A **variable** is a named container that stores a value. Reading a value in, storing it, and using it later are just steps in a sequence — no Java syntax needed yet.



Input box: the value the student types is waiting to be stored.

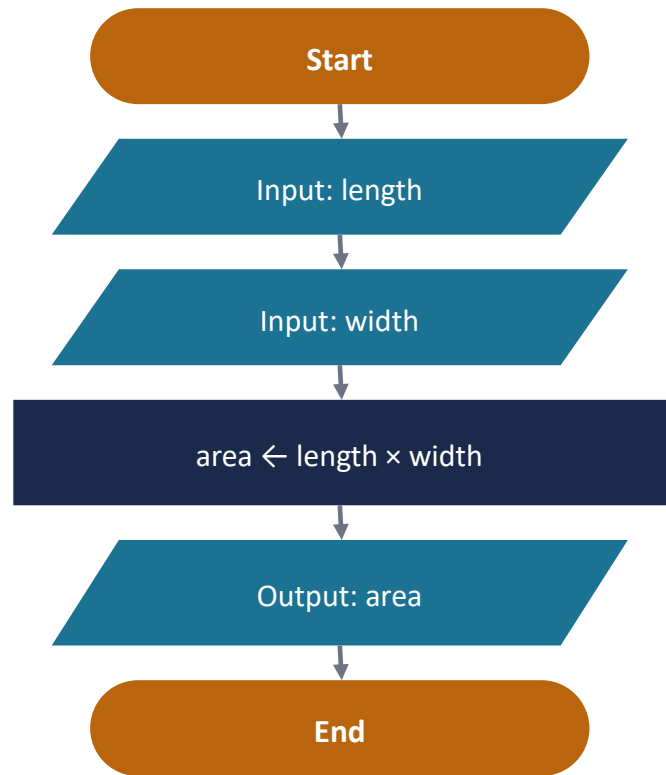


Process box: this is the assignment — “age” now holds that value.



Output box: the variable's current value is used to build a message.

Combining Two Values in One Process

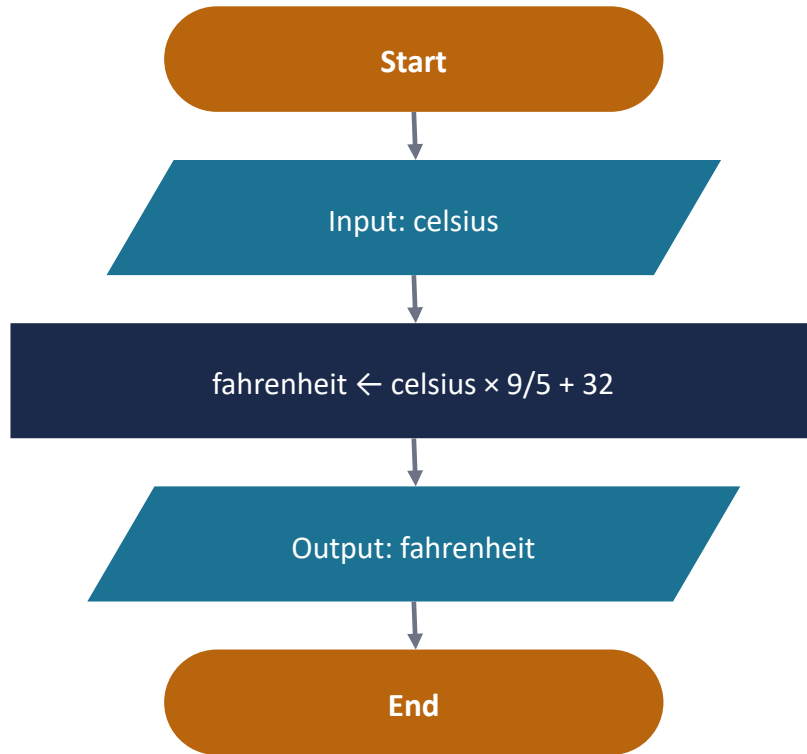


Variables can feed into each other. Here, two inputs are stored, then used together in a single Process step before the result is shown.

Two Input boxes in a row — nothing wrong with that. Each just stores its own value under its own name before the Process step uses both.

Later in Java: `double area = length * width;`

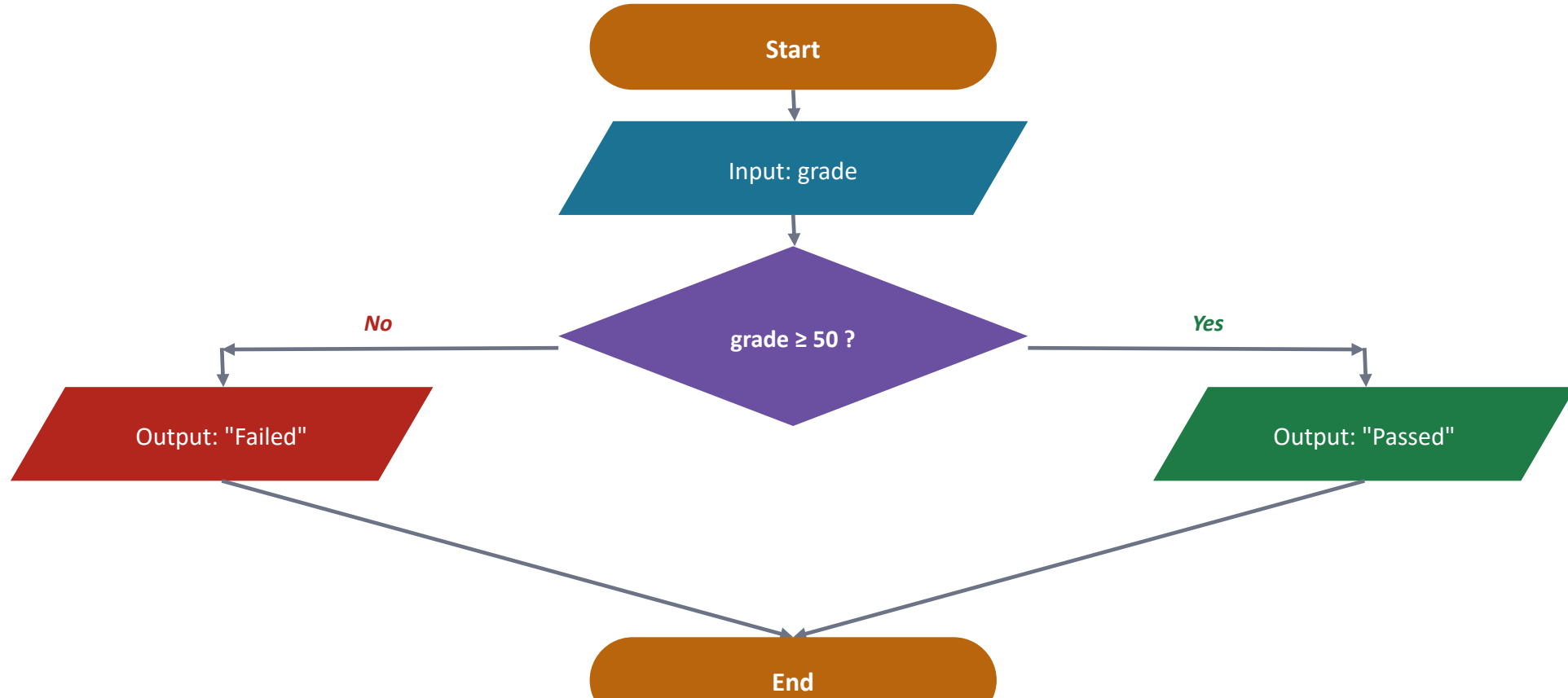
A Process Step Can Apply a Formula



A Process box isn't limited to simple storage. It can transform one variable into a new one using a formula — exactly what an assignment statement will do in Java.

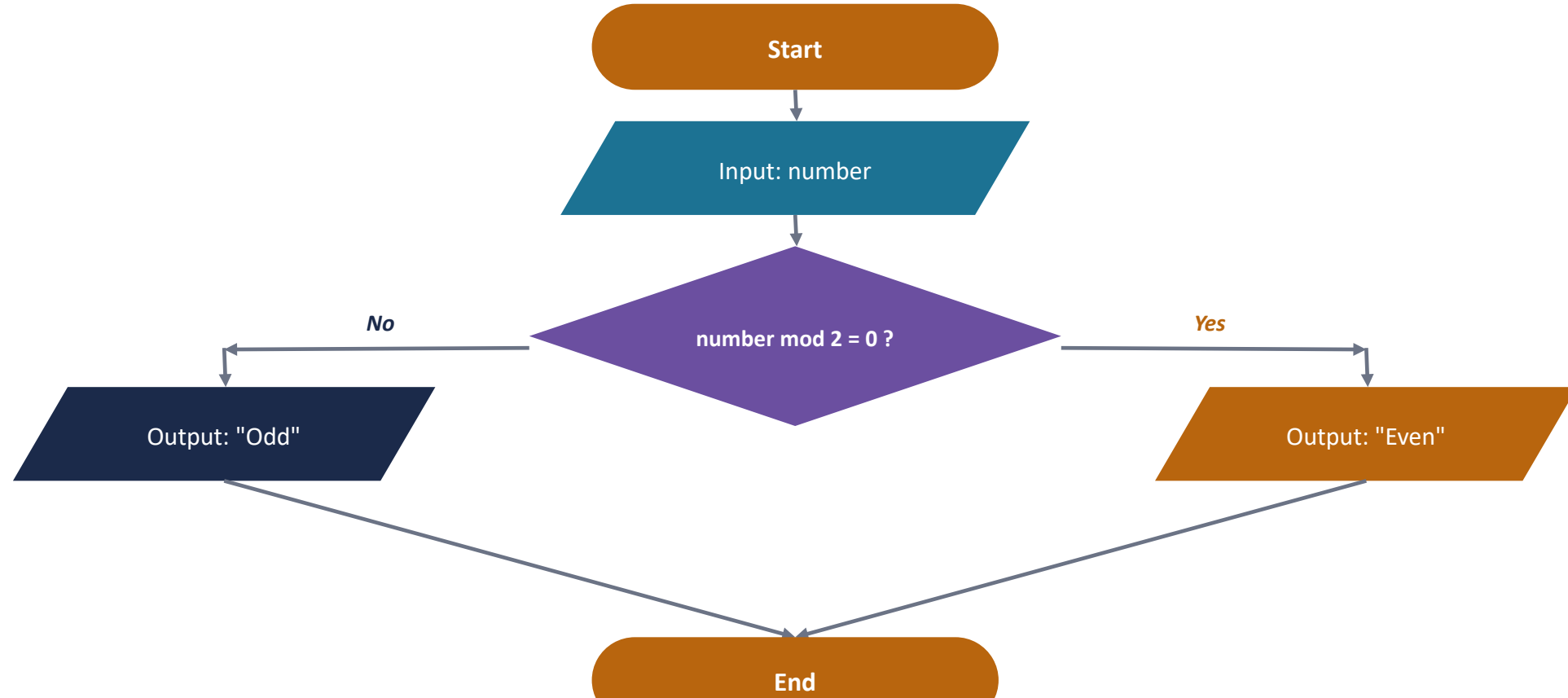
celsius and fahrenheit are two different variables. Reading the formula left to right: compute the right side first, then store it under the name on the left.

The Idea Behind if



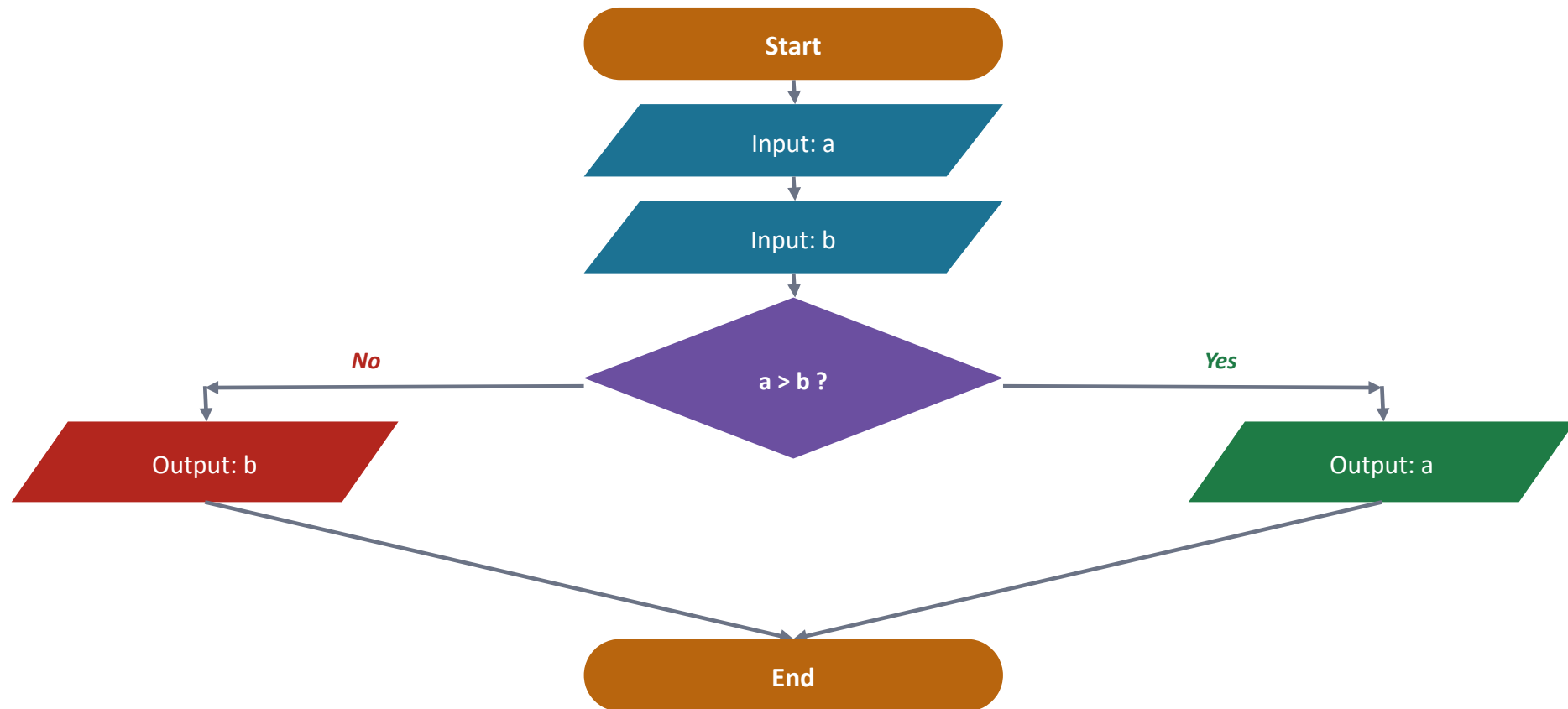
This is exactly what an if / else statement will do in Java: ask a yes/no question, then follow one of two paths.

A Decision Doesn't Need “Pass/Fail”



Any yes/no question works in the diamond — comparisons, remainders, anything that's either true or false.

A Decision Can Compare Two Variables



Two variables can be compared directly in the diamond — no need to involve a third value.

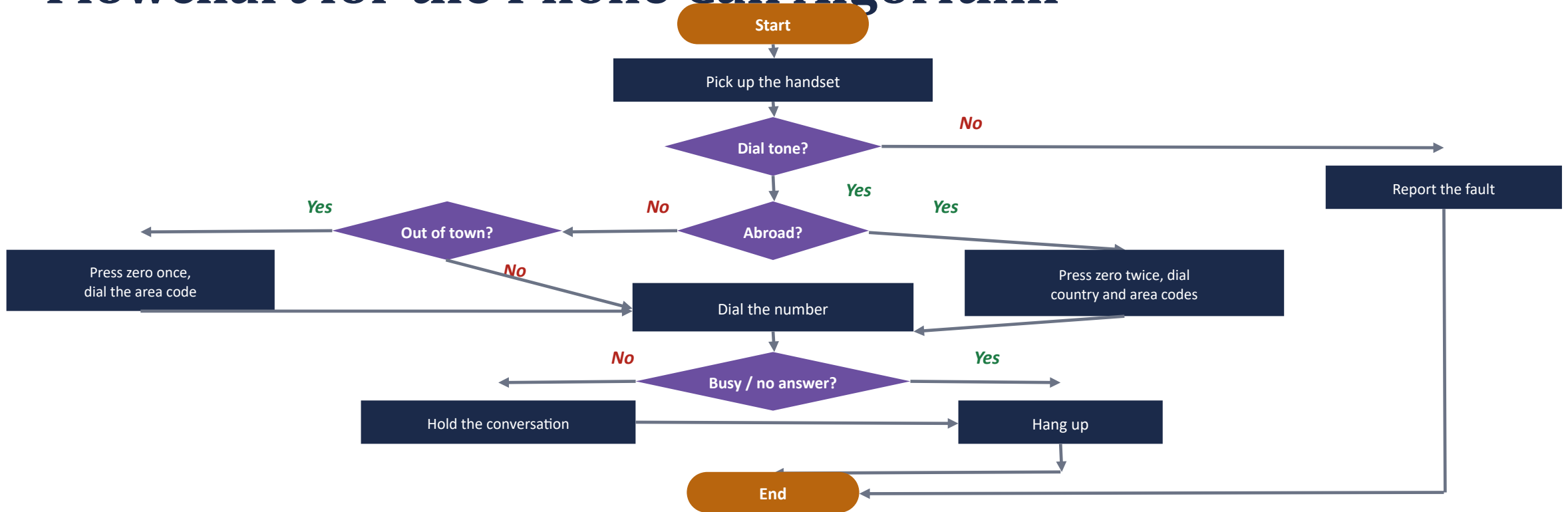
A Real-World Algorithm: Making a Phone Call

Not every algorithm is about numbers. This one describes an everyday task — notice how each “jump to step ...” is exactly what a decision's Yes/No arrow does in a flowchart.

1. Start
2. Pick up the receiver
3. Listen for a dial tone; if there isn't one, report the fault and jump to step 9
4. If the call is international, dial 00, then the country code and area code, then jump to step 6
5. If the call is long-distance, dial 0, then the area code
6. Dial the phone number
7. If the line is busy or nobody answers, jump to step 9
8. Talk
9. Hang up
10. End

Steps run in order; only the “jump to” clauses break that order. For a local call, steps 4 and 5 are both skipped.

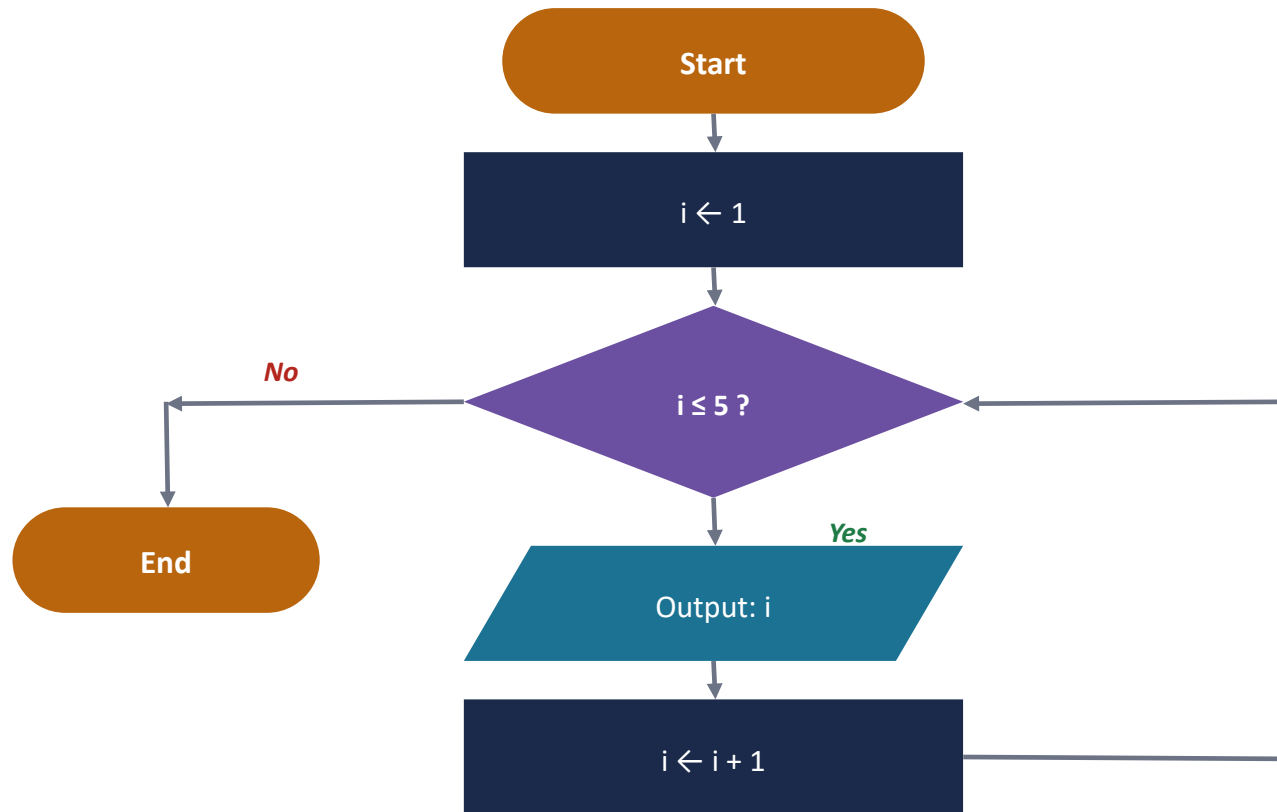
Flowchart for the Phone Call Algorithm



READING THIS CHART

Two decisions run side by side (Abroad? then Out of town?), both feeding “Dial the number.” A bad dial tone skips straight to the end — exactly like the “go to step 9” in the algorithm.

The Idea Behind a Loop



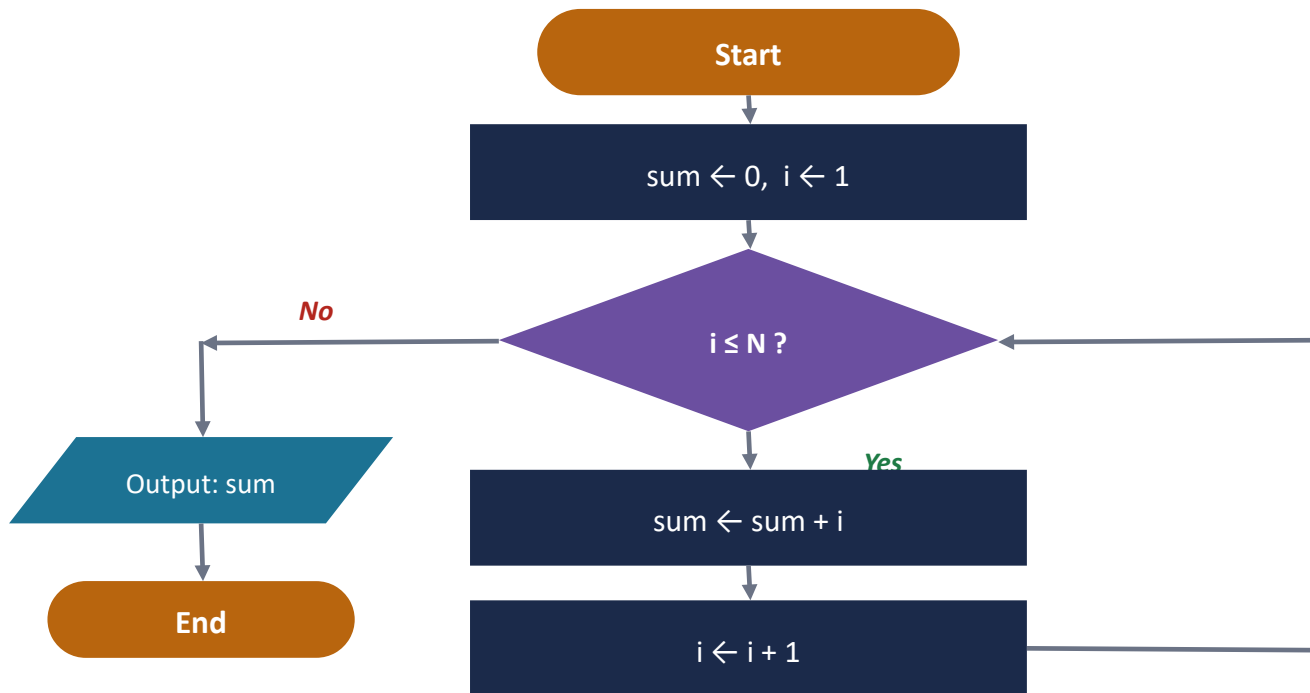
Why it's a loop

A loop is just a decision
with an arrow pointing back to it.

Each time through: check the condition, act, update
— then ask again.

When the condition finally says “No,” the loop ends.

A Loop Can Accumulate a Result



Two variables, two jobs

i counts — it drives the loop and decides when to stop.

sum accumulates — it just keeps growing, one Process step per pass.

Tracing a Loop by Hand

Same flowchart as before — Sum of 1..N, with N = 5. A trace table follows one variable at a time, one pass per row.

Pass	i	i ≤ 5 ?	sum ← sum + i	i ← i + 1
1	1	Yes	0 + 1 = 1	2
2	2	Yes	1 + 2 = 3	3
3	3	Yes	3 + 3 = 6	4
4	4	Yes	6 + 4 = 10	5
5	5	Yes	10 + 5 = 15	6
6	6	No — exit loop	—	—

The loop ran 5 times; sum grew 0 → 1 → 3 → 6 → 10 → 15. The output is 15.

Same Task, Different Algorithms

Problem: add up every whole number from 1 to N. Here are two correct algorithms — they don't cost the same.

Method 1 — Add Them One by One

```
sum ← 0, i ← 1
while i ≤ N do
    sum ← sum + i
    i ← i + 1
end while
output sum
```

The number of steps grows with N. For N = 1,000,000 that's roughly a million additions.

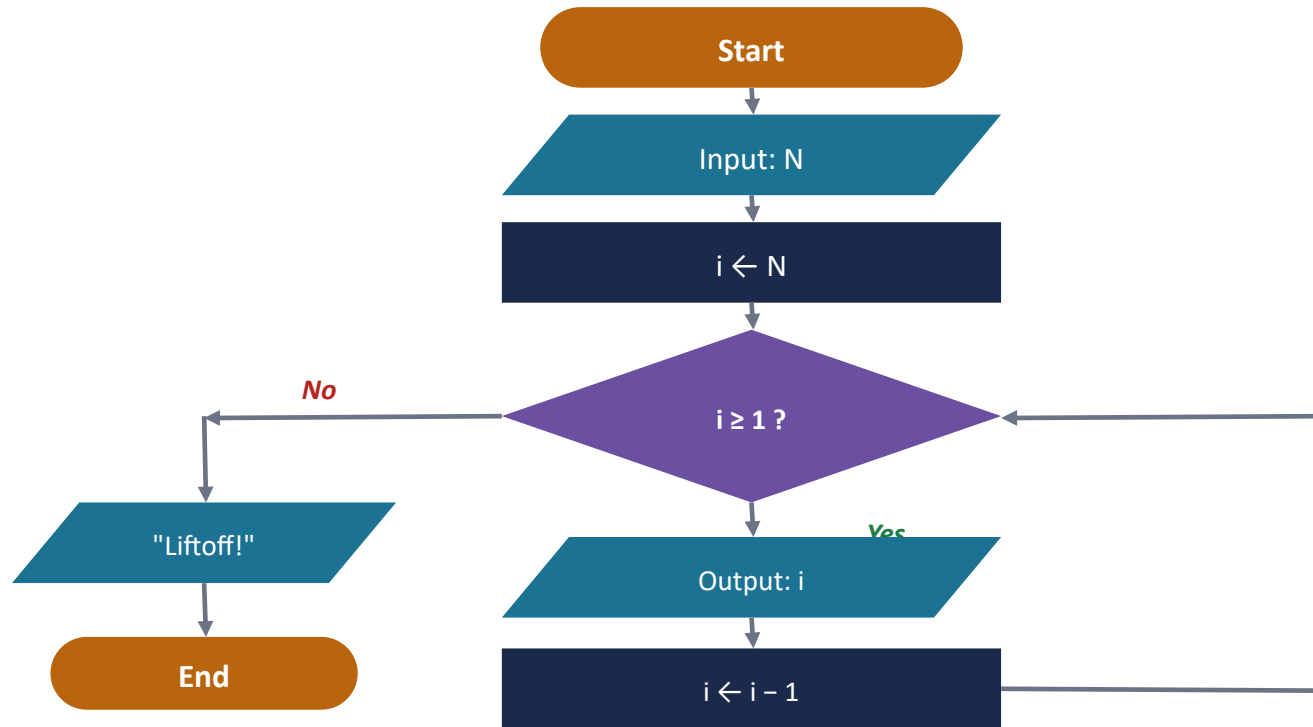
Method 2 — A Closed Formula

```
sum ← N × (N + 1) / 2
output sum
```

The number of steps doesn't depend on N at all — three operations, however large N may be.

Both algorithms give the same correct answer. What separates them is efficiency — an idea we'll return to properly later in the course.

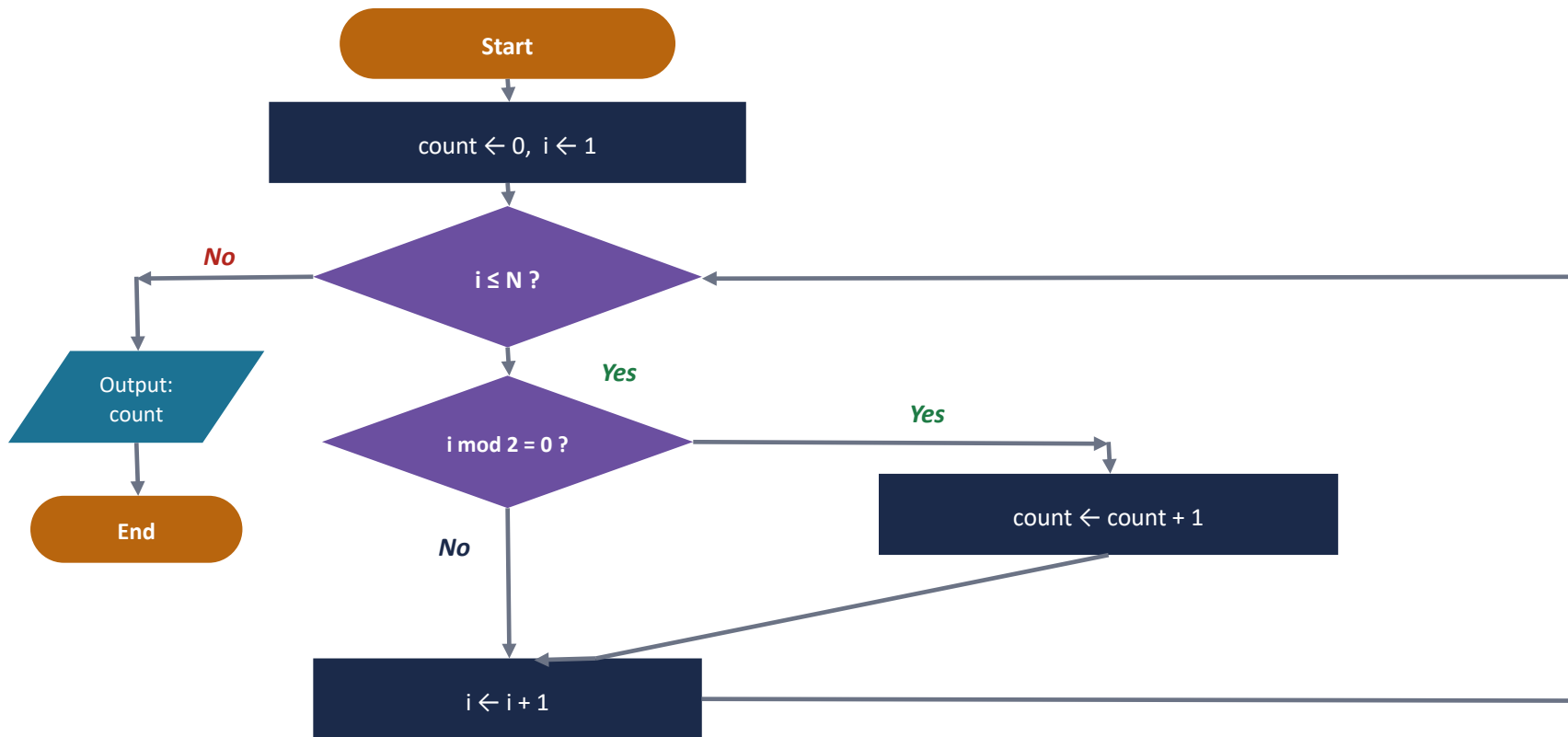
A Loop Can Count Down, Too



Counting doesn't have to go up

The update step subtracts instead of adds — $i \leftarrow i - 1$. The condition still decides when to stop: once i drops below 1, the loop exits and a final message prints.

Sequence + Decision + Loop, Together



Sequence

One process after another — init, check, act, update.



Decision

Two questions here: does the loop continue, and is i even?



Loop

The outer decision, plus an arrow pointing back to it.

Nesting a decision inside a loop is how we'll count, filter, or validate as we go — one flowchart, all three ideas.

From Flowchart to Pseudocode

Pseudocode is the same logic as a flowchart, written as numbered, structured English instead of shapes — the natural stepping stone before real Java syntax.



Process

`sum ← sum + i`

`sum ← sum + i`



Input / Output

`Input: grade`

`input grade`



Decision

`grade ≥ 50 ?`

`if grade ≥ 50 then ... else ...`



Decision + loop-back

`i ≤ N ? (loops back)`

`while i ≤ N do ... end while`

Notice: pseudocode has no semicolons, no curly braces, no strict grammar — just clear, numbered steps a human (or you, in Java, later) can follow exactly.

COMING SOON

From Flowchart to Java (A Preview)



Process

```
int age = 20;
```

An assignment statement



Input / Output

```
System.out.println(...)
```

Reading or printing a value



Decision

```
if (grade >= 50) { ... } else { ... }
```

A branch — exactly one path runs



Decision + loop-back

```
while (i <= N) { ... }
```

The same question, asked again and again

We'll get here once we've covered Java's syntax for variables, if, and loops — the flowchart logic won't change, only how we write it.

Summary & What's Next

- ✓ Numbers are stored in binary — divide-by-2 converts decimal to binary, place values convert it back.
- ✓ Four symbols cover almost every flowchart: terminal, process, decision, input/output.
- ✓ A variable is a named box for a value — setting it, combining it, or transforming it is just a process step.
- ✓ A decision is a yes/no question that sends the flow down one of two paths (the idea behind if).
- ✓ A loop is a decision with an arrow pointing back to itself — traced by hand with a table, one pass per row.
- ✓ Pseudocode writes that same logic as numbered English — the last stop before real Java syntax.

Next: we put these flowcharts into Java syntax — variables, if / else, and while / for loops.